Decision Tree Classifier

In [1]:
```
#step 1: import all required libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
```

In [2]:
```
df=pd.read_csv('playgolf_data.csv')
df1=df.copy()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   Outlook      14 non-null     object
 1   Temperature  14 non-null     object
 2   Humidity     14 non-null     object
 3   Wind         14 non-null     object
 4   PlayGolf     14 non-null     object
dtypes: object(5)
memory usage: 688.0+ bytes
```

In [3]:
```
df
```

Out[3]:

	Outlook	Temperature	Humidity	Wind	PlayGolf
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rainy	Mild	High	Weak	Yes
4	Rainy	Cool	Normal	Weak	Yes
5	Rainy	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rainy	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rainy	Mild	High	Strong	No

In [4]:
```
from sklearn.preprocessing import LabelEncoder

# Encode features
le = LabelEncoder()
#label_encoders = {}
for col in df.columns:
    df[col] = le.fit_transform(df[col])
    df[col]=df[col].astype('category')

print(df)
df.info()
```

```
   Outlook Temperature Humidity Wind PlayGolf
0        2           1        0    1        0
1        2           1        0    0        0
2        0           1        0    1        1
3        1           2        0    1        1
4        1           0        1    1        1
5        1           0        1    0        0
6        0           0        1    0        1
7        2           2        0    1        0
8        2           0        1    1        1
9        1           2        1    1        1
10       2           2        1    0        1
11       0           2        0    0        1
12       0           1        1    1        1
13       1           2        0    0        0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Outlook      14 non-null     category
 1   Temperature  14 non-null     category
 2   Humidity     14 non-null     category
 3   Wind         14 non-null     category
 4   PlayGolf     14 non-null     category
dtypes: category(5)
memory usage: 834.0 bytes
```

In [5]:
```
#divide X and y variables
X = df.drop(['PlayGolf'], axis=1)
y = df['PlayGolf']
```

In [6]:
```
# split X and y into training and testing sets

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.33,
```

In [7]:
```
X_train.shape
```

Out[7]:
```
(9, 4)
```

In [8]:
```
# model Building
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(criterion='entropy', random_state=0)
clf = clf.fit(X, y)
```

In [9]:
```
#predictions
y_pred=clf.predict(X_test)
y_pred
```

Out[9]:
```
array([1, 1, 0, 1, 0], dtype=int64)
```

In [10]:
```
#performance Evaluation
from sklearn.metrics import accuracy_score,classification_report,confusion_m
accuracy_score(y_test,y_pred)
```

Out[10]:
```
1.0
```

In [11]:
```
print(classification_report(y_test,y_pred))
```

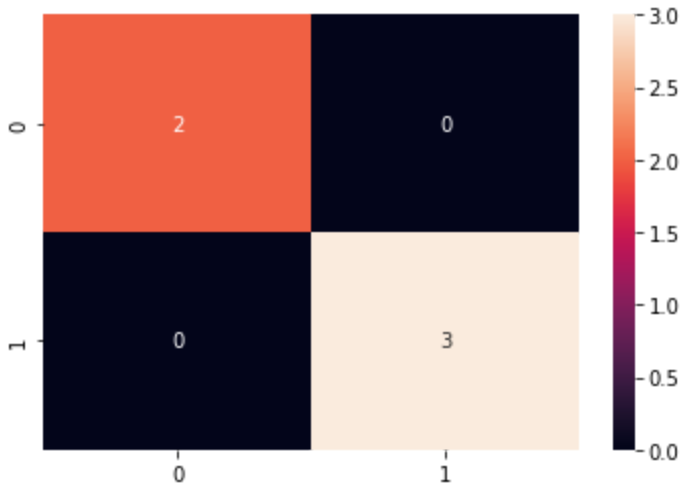
```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         3

    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5
```

In [12]:
```
    import seaborn as sns
sns.heatmap(data=confusion_matrix(y_test,y_pred),annot=True)
```

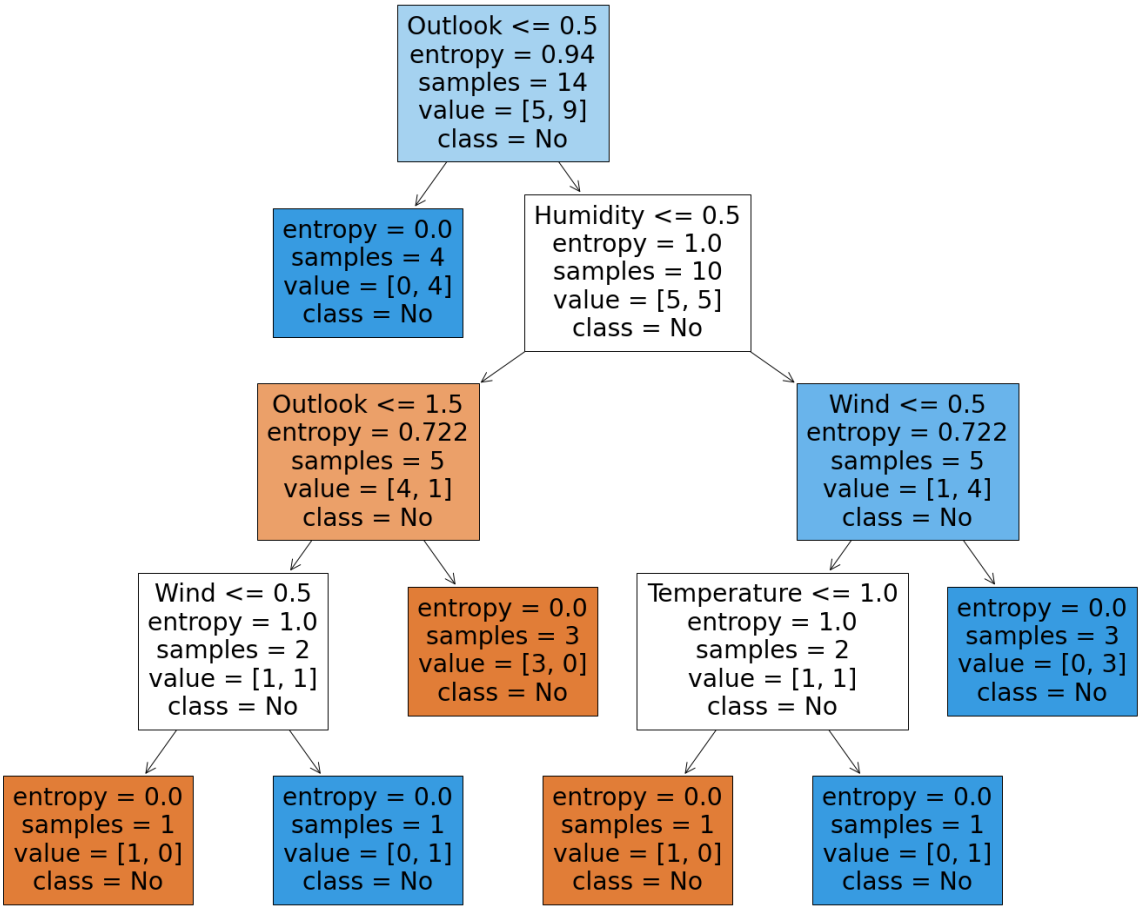
Out[12]:
```
<Axes: >
```



In [13]:
```
from sklearn import tree
```

In [14]:

```
fig = plt.figure(figsize=(25,20))
_ = tree.plot_tree(clf,
                   feature_names=df1.columns,
                   class_names=df1['PlayGolf'],
                   filled=True)
```



In [15]:
```
text_representation = tree.export_text(clf)
print(text_representation)
```

```
|--- feature_0 <= 0.50
|   |--- class: 1
|--- feature_0 >  0.50
|   |--- feature_2 <= 0.50
|   |   |--- feature_0 <= 1.50
|   |   |   |--- feature_3 <= 0.50
|   |   |   |   |--- class: 0
|   |   |   |--- feature_3 >  0.50
|   |   |   |   |--- class: 1
|   |   |--- feature_0 >  1.50
|   |   |   |--- class: 0
|   |--- feature_2 >  0.50
|   |   |--- feature_3 <= 0.50
|   |   |   |--- feature_1 <= 1.00
|   |   |   |   |--- class: 0
|   |   |   |--- feature_1 >  1.00
|   |   |   |   |--- class: 1
|   |   |--- feature_3 >  0.50
|   |   |   |--- class: 1
```

With GINI INDEX

In [16]:
```
# model Building
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(criterion='gini', random_state=0)
clf = clf.fit(X, y)
```

In [17]:
```
y_pred=clf.predict(X_test)
y_pred
```

Out[17]:
```
array([1, 1, 0, 1, 0], dtype=int64)
```

In [18]:
```
#performance Evaluation
from sklearn.metrics import accuracy_score,classification_report,confusion_m
accuracy_score(y_test,y_pred)
```

Out[18]:
```
1.0
```

In [19]:
```
print(classification_report(y_test,y_pred))
```

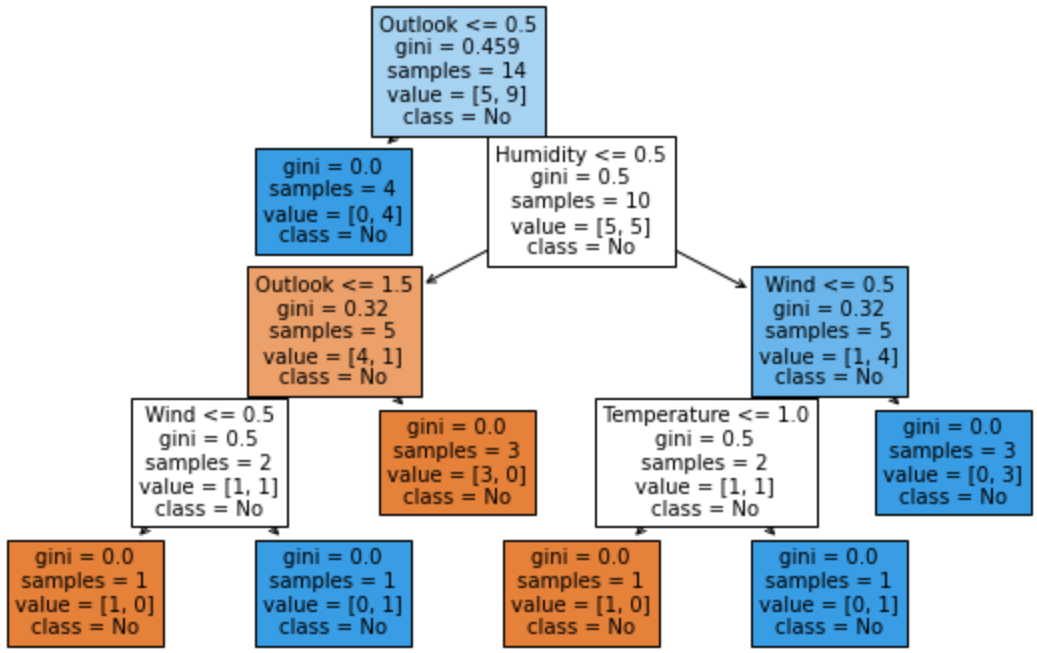
```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         3

    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5
```

In [20]:

```
from sklearn import tree
plt.figure(figsize=(10,6))
tree.plot_tree(clf, feature_names=df1.columns, class_names=df1['PlayGolf'],
plt.show()
```



In [ ]:

```

```

In [ ]:

```

```